

Quantum Computing

Chapter 00: Introduction

Prachya Boonkwan

Version: August 3, 2026

Sirindhorn International Institute of Technology
Thammasat University, Thailand

License: CC-BY-NC 4.0

Who? Me?

- Nickname: [Arm](#) (P'/N' Arm, etc.)
- Born: Aug 1981
- Work
 - Researcher at NECTEC 2005-2024
 - Lecturer at SIIT, Thammasat University 2025-now
- Education
 - B.Eng & M.Eng in Computer Engineering, Kasetsart University, Thailand
 - Obtained Ministry of Science and Technology Scholarship of Thailand in early 2008
 - Did a PhD in Informatics (AI & Computational Linguistics) at University of Edinburgh, UK from 2008 to 2013



Table of Contents

1. Course Outline
2. Classical vs. Quantum Computers
3. Quantum Mechanics
4. Quantum Operations
5. Quantum Algorithms
6. Quantum-Suitable Problems
7. Conclusion

Course Outline

Course Outline

First Half

- Lec 1: Introduction
- Lec 2: Complex algebra and matrices
- Lec 3: Quantum logic gates
- Lec 4: Clifford gates
- Lec 5: Non-Clifford gates
- Lec 6: State measurement
- Lec 7: Deutsch-Jozsa algorithm and phase kickback technique
- Midterm exam

Second Half

- Lec 8: Quantum noisy channels
- Lec 9: Grover's search, Fourier transform, and phase estimation
- Lec 10: Shor's algorithm
- Lec 11: Quantum simulation
- Lec 12: Variational quantum algorithms
- Lec 13: Quantum machine learning
- Lec 14: Quantum optimization
- Final exam

Grading Scheme

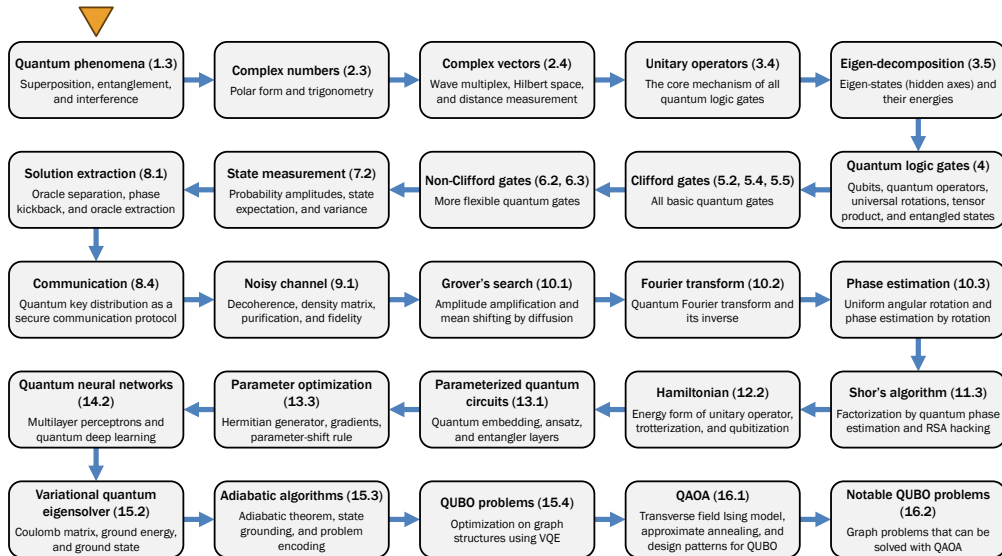
- Midterm and final exams
 - Open-slides, open-book, and open-calculator
 - Majority of my questions are based on discussion and reasoning, while only a few are calculation-oriented (i.e. virtually giving free scores)
- Assignments
 - Two take-home assignments based on calculation and quantum programming on simulators
- No-F's policy: minimum grade is **D+**
- Github repository for the course is <https://github.com/kaamanita/quantum-computing>
- Odorless foods and drinks are allowed in the class due to proximity to lunch time

Items	Proportions
Midterm	35%
Final	35%
Assignments	20%
Attendance	10%
TOTAL	100%

My Forthcoming Book

“Quantum Computing: A Gentle Introduction for AI Engineers”
(Boonkwan, 2026)

Quantum Topics You Really Need to Know



Classical vs. Quantum Computers

What the (quantum) fuss is all about?

- Hearsay: “Quantum computer is immensely faster than classical computer!”
 - Shor’s algorithm for factoring an integer n in $O((\log n)^3)$ time complexity, making RSA code decryption feasible in polynomial time
 - Quantum supremacy vs. quantum advantage
- Quantum computers
 - Perform computation by manipulating state superposition and entanglement in-memory with quantum-mechanical phenomena
 - Are susceptible to noise due to quantum interference
 - Can solve some polynomial-time and non-polynomial-time problems with bounded error
- The notion of classical computer refers to modern-day computers that rely on binary data

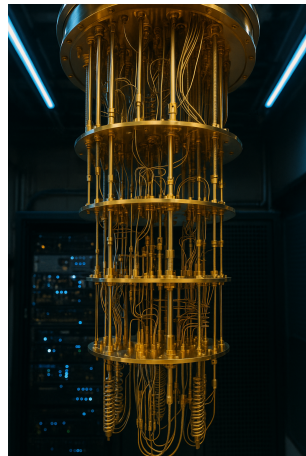


Figure 1: Quantum computer

Classical Computer

- Data: Stored as binary numbers and manipulated by electronic circuits
- Processing: Logical and arithmetic operations, data transfer, and type conversion
- Representation: Only one item out of a vast realm of all possible configurations!
- Limit: We cannot intrinsically process multiple input items in parallel

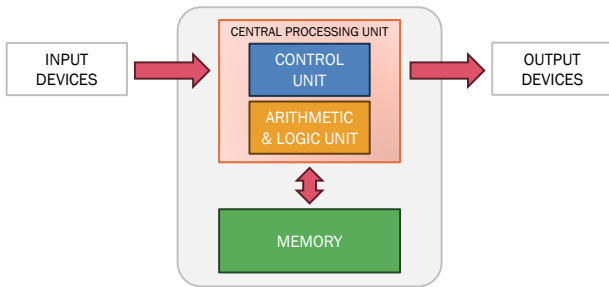


Figure 2: von Neumann architecture of classical computers

Bitstring

- Binary number is a number expressed in the base-2 numeral system: 0 and 1
- Bitstring (or binary representation) of integer N is denoted by $\mathbb{B}_L(N)$, where L is the length of such bitstring, e.g.

$$\mathbb{B}_8(33) = 00100001_2$$

- We can convert a bitstring to a decimal number by multiplying each k -th digit (from the right) with its decimal equivalent 2^{k-1} and summing these multiplicands, e.g.

$$\begin{aligned} 00100001_2 &= 1 \times 2^5 + 1 \times 2^0 \\ &= 33_{10} \end{aligned}$$

- We can also convert an integer to a bitstring by continued division as follows
 1. Find the closest exponent 2^k to N
 2. Subtract 2^k from N
 3. Repeat until the subtraction becomes zero
- Example: We convert 30_{10} to 11110_2 by

$$\begin{aligned} 30 &= 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 \\ &\quad + 1 \times 2^1 + 0 \times 2^0 \end{aligned}$$

2	30	14	6	2	0
	-16	-8	-4	-2	
	2^4	2^3	2^2	2^1	

Table 1: Conversion from 30_{10} to 11110_2

Logical Operations

- AND returns 1 iff both operands are 1

$$0011 \wedge 0101 = 0001$$

- OR returns 1 iff at least one operand is 1

$$0011 \vee 0101 = 0111$$

- NOT returns the opposite value

$$\overline{01} = 10$$

- XOR returns 1 iff only one operand is 1

$$0011 \oplus 0101 = 0110$$

- These abstract logical operations lay the foundation of data processing in classical computing, where conditional branching, jumping, and looping are predominant
- In quantum computing, these operations will be emulated by linear transformation

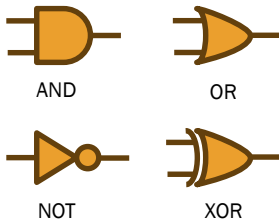


Figure 3: Logical operations

Limitations of Classical Computing

- Classical computing is linearly scalable, due to sequential logical processing of discrete states
- Parallelism in classical computing accelerates sequential processing with multiple computing cores

```
import multiprocessing as mp

def factorial(x):
    result = 1
    for i in range(2, x+1):
        result *= i
    return result

items = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
pool = mp.Pool(processes=4)
results = pool.map(factorial, items)
print(results)
```

- Classical parallelism struggles on looping with exponential iterations, resulting in $O(2^N)$ time complexity
- In quantum computing, multiple discrete values are represented as a superposition of bitstrings

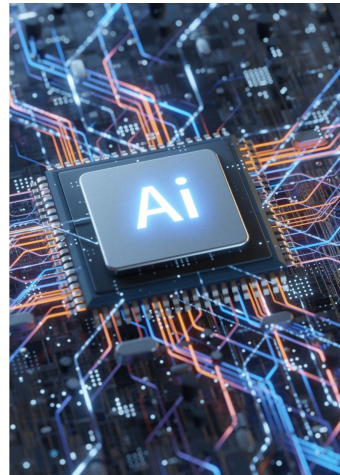
$$\{0000, 0001, \dots, 1111\} \Rightarrow \{0,1\}^4$$

Then the quantum operations are done on this state superposition

- The quantum data processing is based on complex algebra (i.e. complex vectors, complex matrices, and measurement)

Modern-Day AI

- High demand of parallel computation with graphical processing units (GPUs)
- Limitations of high performance computing
 - **Power wall:** Performance gain of transistor-based CPUs are limited by thermal ceiling
 - **Memory wall:** High bandwidth memory still lags behind when compared to the CPUs
 - **Network bottleneck:** Data synchronization between computing units are relatively very slow
 - **Quantum tunneling:** Electrons start to leak through insulation across transistors causing errors & heat



Quantum Computing

- Nondeterminism: probabilistic information processing via quantum mechanics (but we don't have to deal with it)

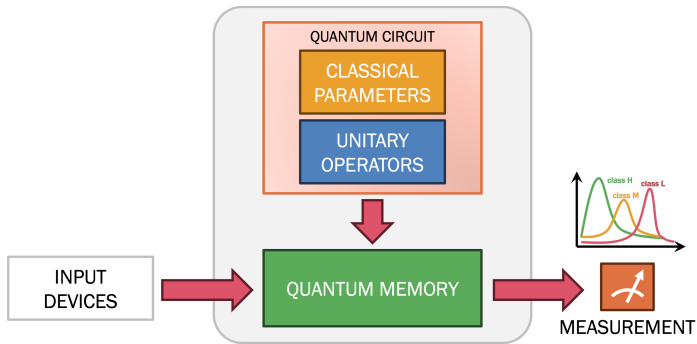


Figure 4: Quantum architecture

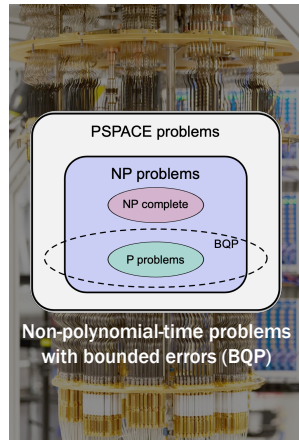


Figure 5: NP-time problems with bounded errors (BQP)

Quantum Mechanics

Quantum Memory

- Qubit: Each bit of information is stored as the momentum spin of a particle
- Information processing is to manipulate these qubits via physical interaction e.g. laser beam (trapped ions) or light interference (photons)

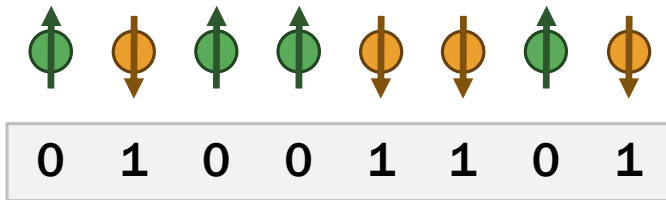


Figure 6: A classical bitstring represents the system's current state.

State Measurement

- Momentum spins are destroyed when we read the system's current state

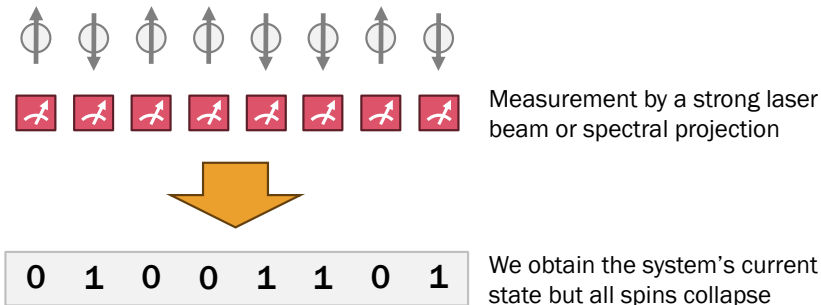


Figure 7: State measurement

Phenomenon 1: Superposition

- A quantum system can exist in multiple states (with probabilities) in parallel

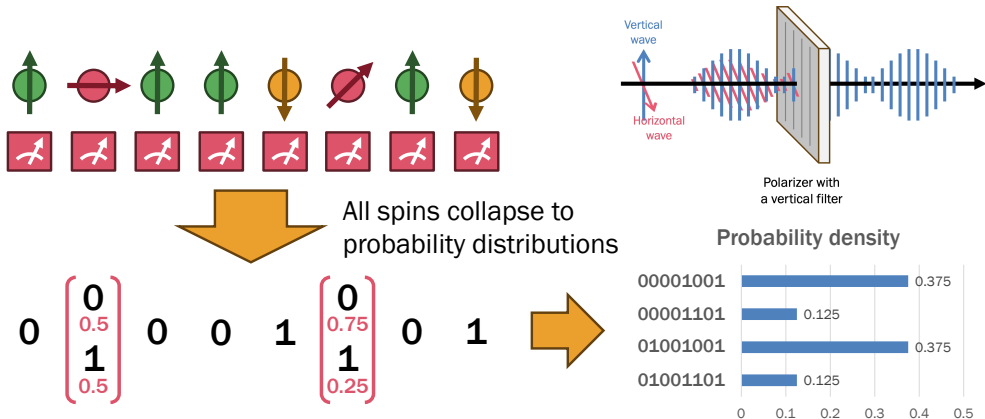


Figure 8: Quantum superposition

Quantum State and Probability Amplitudes

- Quantum state: the system's configuration of momentum spins for each qubit
- Three dipoles (0/1, +/−, and +i/−i) represent the 3D momentum spin of one qubit

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

where complex numbers a and b are probability amplitudes of $|\psi\rangle$ collapsing to 0 and 1

- Probability density of $|\psi\rangle$ collapsing to 0 or 1:

$$P(0|\psi) = a^2$$

$$P(1|\psi) = b^2$$

- Dirac's notation: $|\psi\rangle$ reads 'ket'-psi

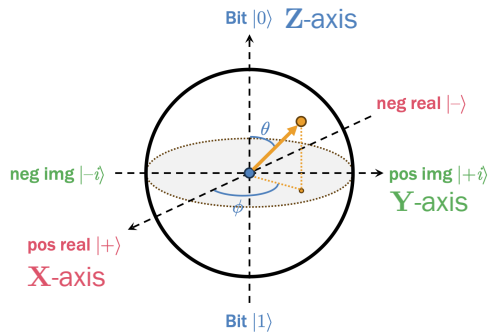


Figure 9: Bloch sphere

Multi-Qubit System

- Tensor product of quantum states

$$\begin{aligned} & (a_1|0\rangle + b_1|1\rangle) \otimes (a_2|0\rangle + b_2|1\rangle) \\ &= a_1a_2|00\rangle + a_1b_2|01\rangle + b_1a_2|10\rangle + b_1b_2|11\rangle \end{aligned}$$

where $\otimes$ denotes concatenation

- Factorization of quantum state

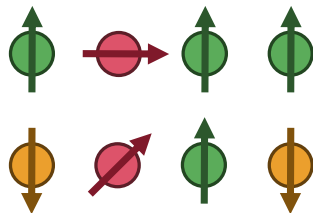
$$\begin{aligned} & \frac{\sqrt{3}}{2\sqrt{2}}|00001001\rangle + \frac{1}{2\sqrt{2}}|00001101\rangle \\ & + \frac{\sqrt{3}}{2\sqrt{2}}|01001001\rangle + \frac{1}{2\sqrt{2}}|01001101\rangle \end{aligned}$$

Multi-state superposition



$$\begin{aligned} & |0\rangle \otimes \left(\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle \right) \otimes |0\rangle \otimes |0\rangle \\ & \otimes |1\rangle \otimes \left(\frac{\sqrt{3}}{2}|0\rangle + \frac{1}{2}|1\rangle \right) \otimes |0\rangle \otimes |1\rangle \end{aligned}$$

Tensor product



Dirac's Notation

- Single-qubit ket = 2-dim column vector

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad a|0\rangle + b|1\rangle = \begin{bmatrix} a \\ b \end{bmatrix}$$

- L -qubit ket = 2^L -dim column vector

$$|k\rangle = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad \sum_{k=0}^{2^L-1} a_k |k\rangle = \begin{bmatrix} a_0 \\ \vdots \\ a_{2^L-1} \end{bmatrix}$$

- $|k\rangle$ is 1-hot vector at the k -th position

- Single-qubit bra = 2-dim row vector

$$\begin{aligned} (a|0\rangle + b|1\rangle)^* &= \bar{a}\langle 0| + \bar{b}\langle 1| \\ &= \begin{bmatrix} \bar{a} & \bar{b} \end{bmatrix} \end{aligned}$$

- L -qubit bra = 2^L -dim row vector

$$\sum_{k=0}^{2^L-1} a_k \langle k| = \begin{bmatrix} a_0 & \dots & a_{2^L-1} \end{bmatrix}$$

- Bra-ket product = vector similarity

$$\langle k|k\rangle = 1 \quad \langle j|k\rangle = 0$$

where $j \neq k$

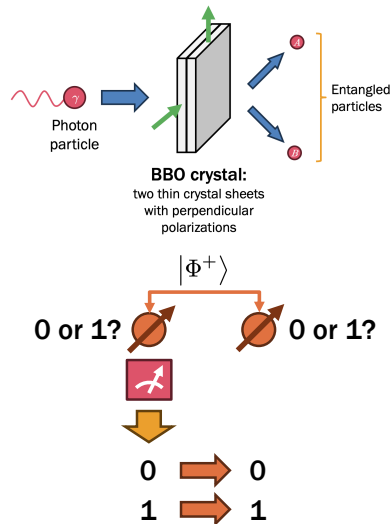
Phenomenon 2: Entanglement

- Multiple particles may become inextricably linked to each other, i.e. they are entangled
- Entangled states are multi-qubit states that cannot be factorized further, e.g. Bell states

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}} \quad |\Phi^-\rangle = \frac{|00\rangle - |11\rangle}{\sqrt{2}}$$

- If one of them collapses, the rest of the pack also collapse instantaneously:

$$P(00|\Phi^+) = \frac{1}{2} \quad P(11|\Phi^+) = \frac{1}{2}$$



Phenomenon 3: Quantum Interference

- If we manipulate the prob amplitude of a qubit, this change ripples to the others
- Probability amplitude can be manipulated in two ways
 - **Amplification:** increasing the prob amplitude
 - **Dampening:** decreasing the prob amplitude
- Quantum interference is the key mechanism of quantum gate design

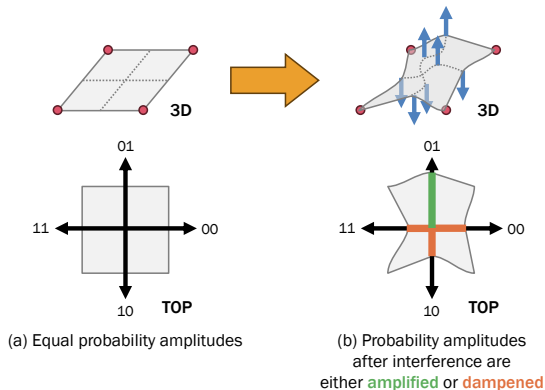


Figure 10: Quantum interference

Quantum Operations

Designing a Quantum Operator/1

- We manipulate the system's quantum state by applying a quantum operator
- [State mapping](#)

$$\begin{array}{c} |\text{input } \psi_1\rangle \mapsto |\text{output } \phi_1\rangle \\ \vdots \\ |\text{input } \psi_K\rangle \mapsto |\text{output } \phi_K\rangle \end{array}$$

- [Quantum operator](#)

$$U = \sum_{k=1}^K |\text{output } \phi_k\rangle \langle \text{input } \psi_k|$$

- Reminder: $\langle \psi| = |\psi\rangle^*$

- Generally, we use pure states as inputs ψ_k
- The quantum operator applies [interference](#) on the target qubits

$$\begin{aligned} & U |\text{state } \xi\rangle \\ &= \left(\sum_{k=1}^K |\text{output } \phi_k\rangle \langle \text{input } \psi_k| \right) |\text{state } \xi\rangle \\ &= \sum_{k=1}^K |\text{output } \phi_k\rangle \underbrace{\left(\langle \text{input } \psi_k| |\text{state } \xi\rangle \right)}_{\text{similarity}} \\ &= \sum_{k=1}^K |\text{output } \phi_k\rangle \langle \text{input } \psi_k| \text{state } \xi \rangle \end{aligned}$$

Designing a Quantum Operator/2

- Example: NOT operator has state mapping

$$|0\rangle \mapsto |1\rangle \quad |1\rangle \mapsto |0\rangle$$

The quantum operator for NOT is

$$\begin{aligned} \mathbf{X} &= |1\rangle\langle 0| + |0\rangle\langle 1| \\ &= \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \end{aligned}$$

- Compared with matrix multiplication

$$\begin{aligned} \mathbf{X} \begin{bmatrix} a \\ b \end{bmatrix} &= \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} \\ &= \begin{bmatrix} 0a + 1b \\ 1a + 0b \end{bmatrix} \\ &= \begin{bmatrix} b \\ a \end{bmatrix} \end{aligned}$$

- Compared with state mapping

$$\begin{aligned} \mathbf{X}(a|0\rangle + b|1\rangle) &= a(\mathbf{X}|0\rangle) + b(\mathbf{X}|1\rangle) \\ &= a|1\rangle + b|0\rangle \\ &= b|0\rangle + a|1\rangle \end{aligned}$$

NOT Gate: X

- X gate flips the qubit between $|0\rangle$ and $|1\rangle$
- State mapping

$$|0\rangle \mapsto |1\rangle \quad |1\rangle \mapsto |0\rangle$$

- It imitates logical NOT in classical computing

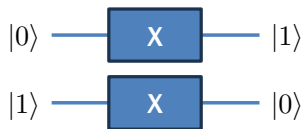
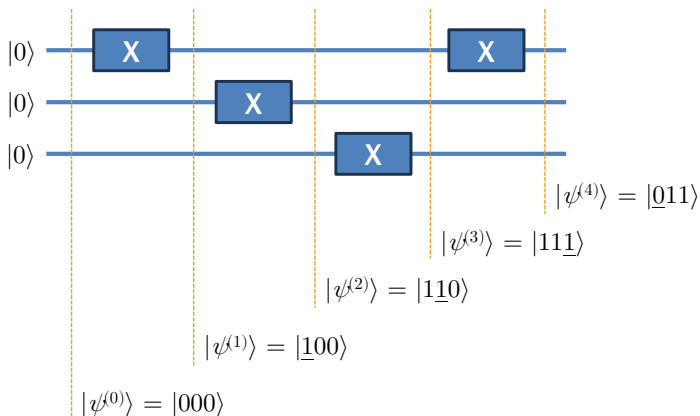


Figure 11: X gate

- Quantum algorithm is a circuit of quantum logic gates



Hadamard Gate: H

- H gate flips between pure states and superpositions

- State mapping

$$|0\rangle \mapsto \underbrace{\frac{|0\rangle + |1\rangle}{\sqrt{2}}}_{|+\rangle} \quad |1\rangle \mapsto \underbrace{\frac{|0\rangle - |1\rangle}{\sqrt{2}}}_{|-\rangle}$$

- Used for parallel processing

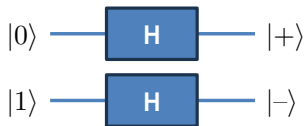
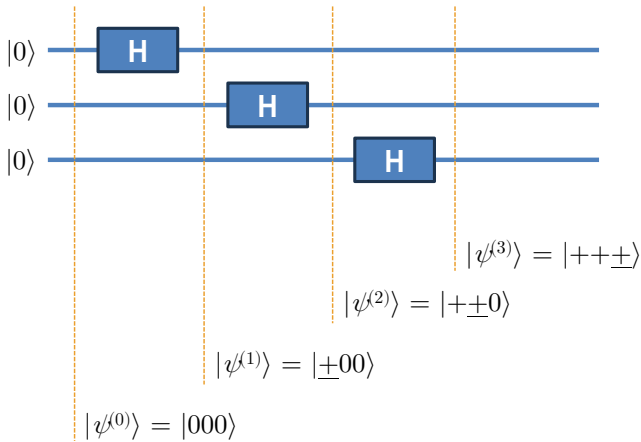


Figure 12: Hadamard gate

- Preparing a superposition of all possible 2^L inputs



Controlled NOT Gate: CNOT

- CNOT gate imitates an if-clause and creates entanglement
- State mapping

$$|0x\rangle \mapsto |0x\rangle \quad |1x\rangle \mapsto |1\bar{x}\rangle$$

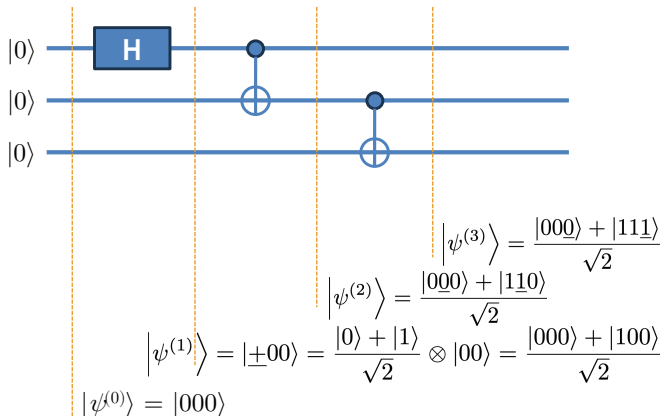
where x is either 0 or 1

- Last qubit is the output equivalent to classical XOR gate



Figure 13: Controlled NOT gate

- Preparing an entangled state with H and CNOT gates



Phase Shift Gate: S

- S_gate flips between real and complex numbers on $|1\rangle$
- State mapping

$$|0\rangle \mapsto |0\rangle \quad |1\rangle \mapsto i|1\rangle$$

- Imaginary number $i = \sqrt{-1}$ is used as soft feature in quantum machine learning

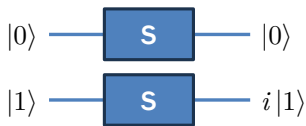
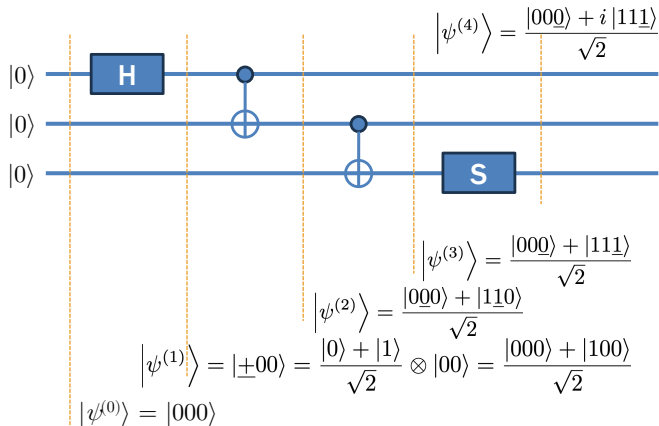


Figure 14: Phase shift gate

- Preparing an entangled state with complex probability amplitudes



Toffoli Gate: CCNOT

- CCNOT gate imitates an if-clause with two control qubits
- State mapping

Both controls: $|11x\rangle \mapsto |11\bar{x}\rangle$

Otherwise: $|xyz\rangle \mapsto |xyz\rangle$

where x and y are binary flags

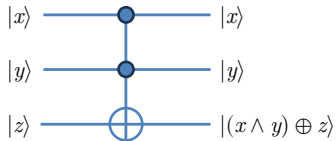
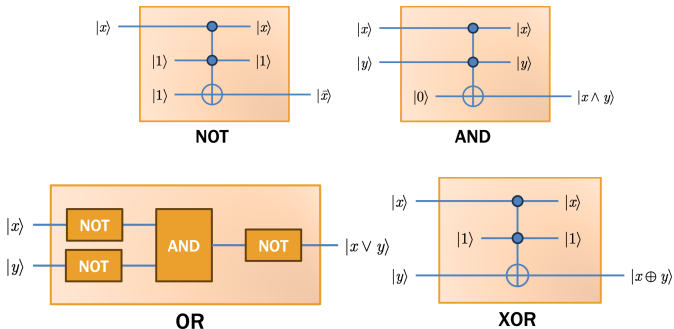


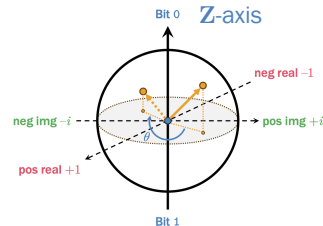
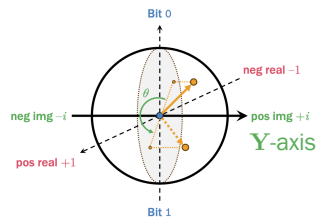
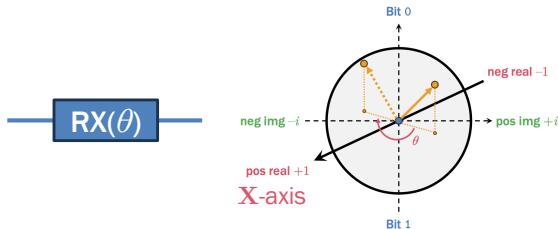
Figure 15: Toffoli gate

- CCNOT gate imitates all classical logic gates, allowing massive parallelism on classical programs



Rotation Gates: R_X , R_Y , and R_Z

- Rotation gates rotate a qubit around the specified eigenbases
- Rotation alters the probability densities used in quantum machine learning
- X-rotation $R_X(\theta)$, Y-rotation $R_Y(\theta)$, and Z-rotation $R_Z(\theta)$, where θ is a model parameter



Controlled Gate: $C^{|k\rangle}U$

- Extension of CNOT, where control qubits are $|k\rangle$

- State mapping of $C^{|k\rangle}U$

- If controls $|x_1, \dots, x_L\rangle = |k\rangle$

$$|x_1, \dots, x_L, y\rangle \mapsto |x_1, \dots, x_L\rangle \otimes U|y\rangle$$

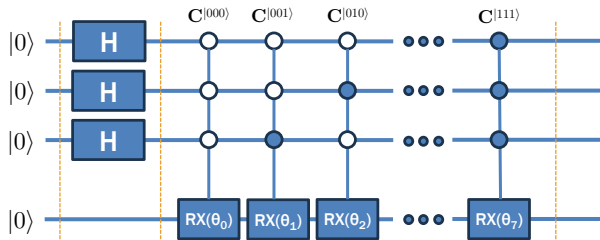
- Otherwise

$$|x_1, \dots, x_L, y\rangle \mapsto |x_1, \dots, x_L, y\rangle$$

- This is frequently used in Fourier transform and phase estimation

Notation:

$\bigcirc$ $c = 0$ $\bullet$ $c = 1$



$$|\psi^{(9)}\rangle = \frac{1}{\sqrt{8}} \sum_{k=0}^7 (|k\rangle \otimes \mathbf{R}\mathbf{X}(\theta_k) |0\rangle)$$

$$|\psi^{(1)}\rangle = |+++0\rangle$$

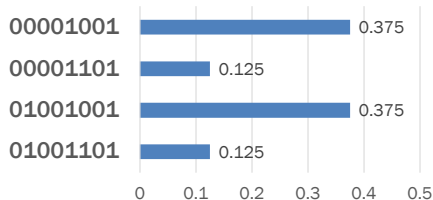
$$|\psi^{(0)}\rangle = |0000\rangle$$

State Measurement

- Probability density: We destructively measure the probability distribution of each pure state by projection (i.e. the quantum state collapses)



Probability density



- Expectation: We non-destructively measure if each k -th qubit leans towards one side of the particular dipoles $\mathbf{U} \in \{\mathbf{X}, \mathbf{Y}, \mathbf{Z}\}$

$$\langle \mathbf{U}^{(k)} \rangle = \langle \psi | \mathbf{U} | \psi \rangle$$



Expectations	Positive (+1)	Negative (-1)
$\langle \mathbf{Z}^{(k)} \rangle$ or Z_k	Bit 0	Bit 1
$\langle \mathbf{X}^{(k)} \rangle$ or X_k	Pos real	Neg real
$\langle \mathbf{Y}^{(k)} \rangle$ or Y_k	Pos img	Neg img

Quantum Algorithms

Oracle Extraction

- Bernstein-Vazirani algorithm
 - Extracting an oracle $\mathbf{s}$ from a black-box function f_s by converting it to U_{f_s}
- Phase kickback technique
 - State mapping of U_{f_s}

Oracle: $|s_{1:N}, y\rangle \mapsto |s_{1:N}, y \text{ cis } \theta\rangle$

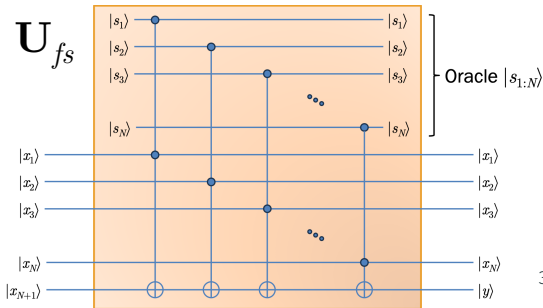
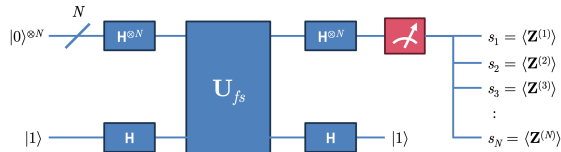
Otw.: $|x_{1:N}, y\rangle \mapsto |x_{1:N}, y\rangle$

- Kickback property

$$U_{f_s} \left(|+\rangle^{\otimes N} \otimes |-\rangle \right) = \left(\bigotimes_{k=1}^N \frac{|0\rangle + (\text{cis } \theta)^{f_{sk}(2^{k-1})} |1\rangle}{\sqrt{2}} \right) \otimes |-\rangle$$

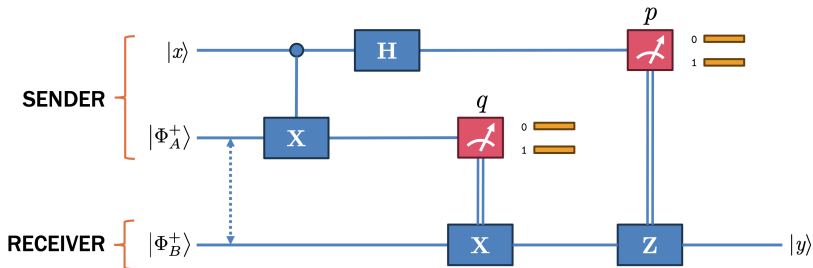
- The last qubit interferes the others

The pure state $\mathbf{s}$ will have the highest probability density



Quantum Teleportation

- Secure communication based on quantum mechanics
- We want to transmit state $|x\rangle = a|0\rangle + b|1\rangle$ with entangled state $\Phi^+ = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$
- How it works:
 - The sender and receiver have $|\Phi_A^+\rangle$ and $|\Phi_B^+\rangle$, respectively
 - The sender runs the quantum teleportation algorithm on the input qubit $|x\rangle$
 - The sender measures p and q and send them over classical communication
 - The receiver uses them to translate $|\Phi_B^+\rangle$ to $|x\rangle$



p	q	State $ y\rangle$
0	0	$a 0\rangle + b 1\rangle$
0	1	$b 0\rangle + a 1\rangle$
1	0	$a 0\rangle - b 1\rangle$
1	1	$-b 0\rangle + a 1\rangle$

Quantum Key Distribution

- Secure communication without actual key transmission, such as [BB84](#)

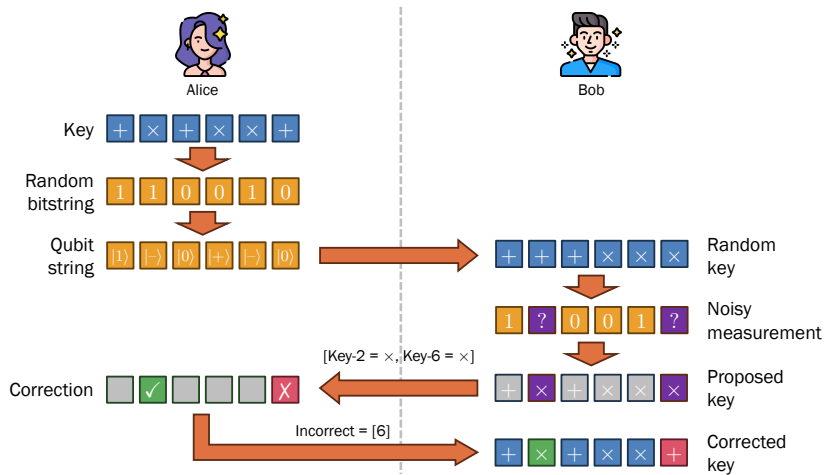


Figure 16: BB84 Protocol

Grover's Search Algorithm

- We can search for 1 of N items w.r.t. the oracle function f with $k^* = \left\lfloor \frac{\pi}{4} \sqrt{N} \right\rfloor$ iterations

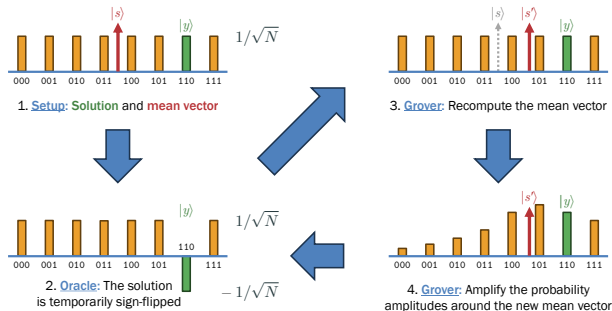


Figure 17: Overview of Grover's algorithm

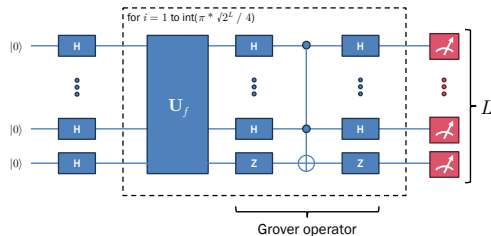
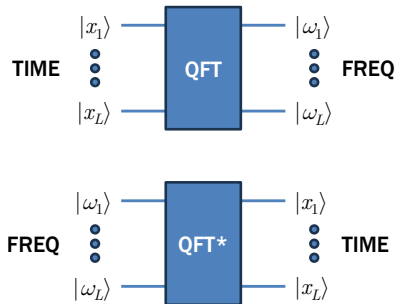
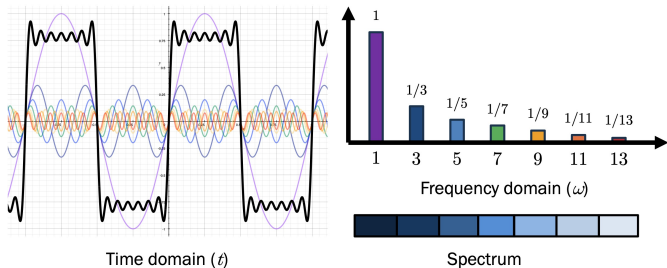


Figure 18: Grover's algorithm. U_f is equivalent to Step 2, while the Grover's operator is equivalent to Steps 3 and 4.

Quantum Fourier Transform

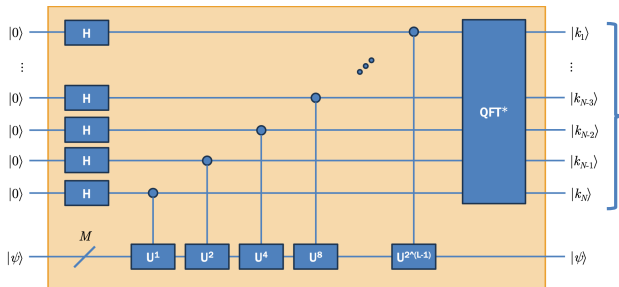
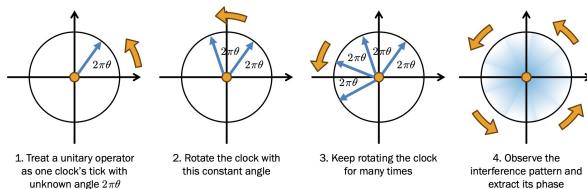
- Translates a time series of N items to a frequency domain, and vice versa, with $O((\log N)^2)$ time complexity



where $L = \log N$

Quantum Phase Estimation

We can extract a phase $2\pi\theta$ of a black-box algorithm $\mathbf{U}$ by rotating it in different frequencies and observe the resonance frequency



$\text{QPE}_N(\mathbf{U})$

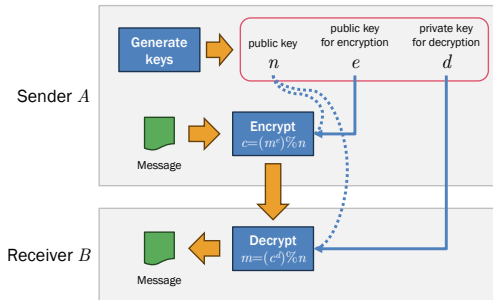
Measure for $k^* = \text{pure state with max prob}$

Phase $\theta = k^*/2^N$

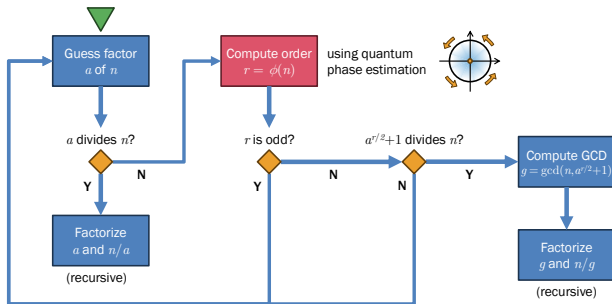
Quantum-Suitable Problems

Shor's Algorithm

- RSA cryptosystem
 - Public and two private keys (n, e, d)
 - Encryption: $c = (m^e) \% n$
 - Decryption: $m = (c^d) \% n$



- Attack with QPE
 - Modulo $\text{Umod } n(a)$ is eqv. to rotation
 - QPE is used to guess the phase
 - Time complexity $O(L^2 \log L)$



Simulation with Trotterization

- Hamiltonian $\hat{H}_U$ is the energy form of quantum circuit U of L qubits

$$U = \text{cis}(-\hat{H}_U)$$

- We can compute $\hat{H}_U$ by

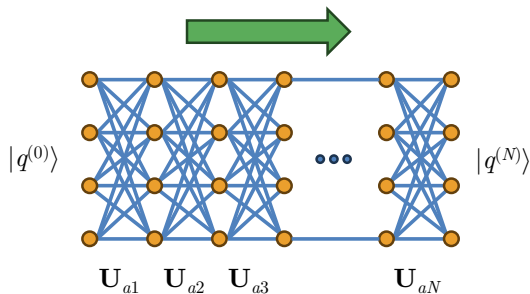
$$\hat{H}_U = \sum_{P \in \{I, X, Y, Z\}^{\otimes L}} [P \times \text{trace}(UP)]$$

where I, X, Y, Z are Pauli matrices

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$
$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

- Trotter-Suzuki decomposition: We can simulate N iterations of U by approximation

$$U^N \approx \lim_{n \rightarrow \infty} \left[\text{cis} \left(-\frac{N}{n} \hat{H}_U \right) \right]^n$$



- Variational quantum algorithm

- Quantum circuit with rotation gates $\mathbf{RX}(\theta)$, $\mathbf{RY}(\theta)$, and $\mathbf{RZ}(\theta)$
- Basic entangler layer (**BEL**)

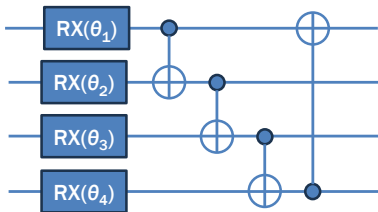


Figure 19: Basic entangler layer

- Quantum embedding $\Lambda(\Theta)$
 - Angle embedding: params for $\mathbf{RX}(\theta_k)$
 - Amplitude embedding: params for each pure state
- Hybrid classical-quantum computing

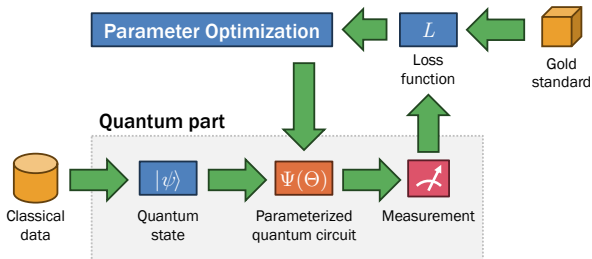
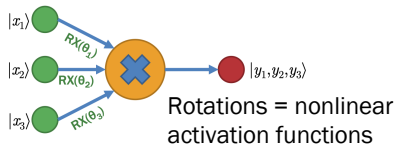


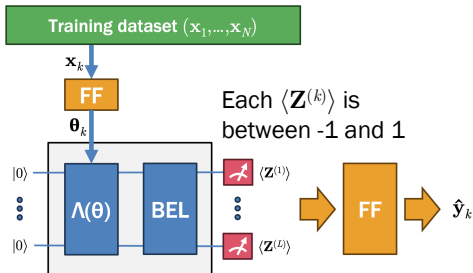
Figure 20: Hybrid system

Quantum Neural Networks

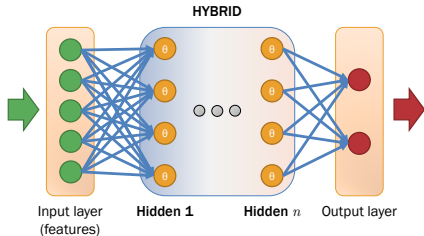
- Quantum perceptron



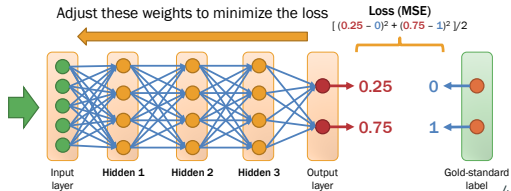
- Quantum multilayer perceptrons



- Quantum deep NNs



- Backpropagation



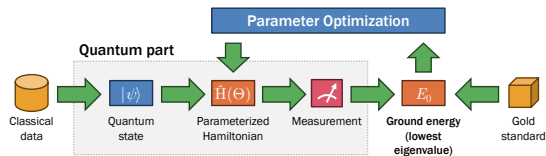
Variational Quantum Eigensolver (VQE)

- We want to minimize a loss Hamiltonian defined on qubit expectations

$$L(\Theta) = f\left(\langle \mathbf{Z}^{(\psi_1)} \rangle, \dots, \langle \mathbf{Z}^{(\psi_L)} \rangle\right)$$

- Parameter shift rule: Computing gradient for SGD by shifting Θ by $\pi/2$

$$\frac{\partial}{\partial \Theta} L(\Theta) = \frac{1}{2} \left(L\left(\Theta + \frac{\pi}{2}\right) - L\left(\Theta - \frac{\pi}{2}\right) \right)$$



- NFT Algorithm: Gradually re-estimating each parameter at timestep t

$$\theta_j^{(t+1)} = -\frac{\pi}{2} - \arctan \frac{y_j^{(t)} - z_j^{(t)}}{2x_j^{(t)} - y_j^{(t)} - z_j^{(t)}}$$

where

$$x_j^{(t)} = L(\Theta^{(t)} | \theta_j = \theta_j^{(t)})$$

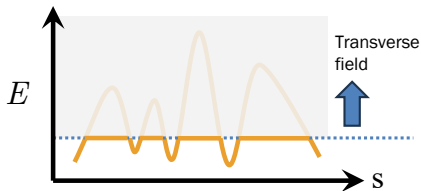
$$y_j^{(t)} = L(\Theta^{(t)} | \theta_j = \theta_j^{(t)} + \frac{\pi}{2})$$

$$z_j^{(t)} = L(\Theta^{(t)} | \theta_j = \theta_j^{(t)} - \frac{\pi}{2})$$

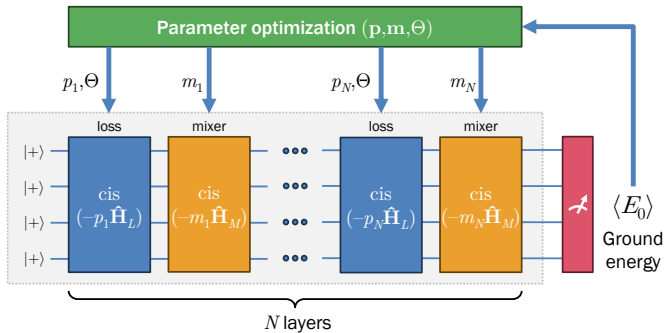
- Both methods converge very fast

Quantum Approximate Optimization Algorithm (QAOA)

- Quantum annealing consists of two steps
 - **Mixer**: explore the transverse field to improve coverage
 - **Loss**: identify the globally optimal point in the explored transverse field



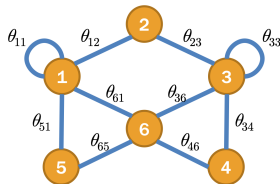
- Objective function = N layers of mixer Hamiltonian $\hat{H}_M$ + loss Hamiltonian $\hat{H}_L$



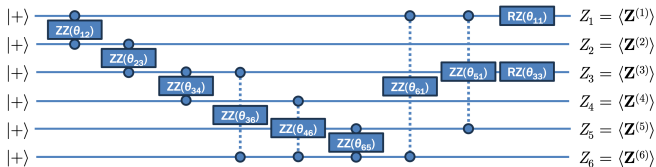
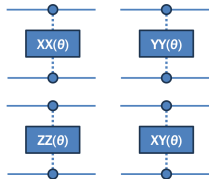
QAOA: Quadratic Unbounded Binary Optimization (QUBO)

- Each edge (i, j) becomes a model parameter θ_{ij}
- Loss function:

$$L = \sum_{(i,i) \in E} \frac{\theta_{ii}}{2} Z_i + \sum_{(i,j) \in E} \frac{\theta_{ij}}{2} Z_i Z_j$$

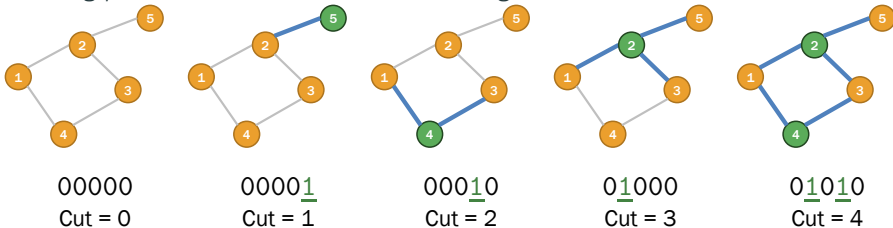


Ising gates rotate both input qubits with the same phase θ



QAOA: Max-Cut as a QUBO Problem

- Graph-coloring problem where the number of edges between two colors is maximum



- Mixer Hamiltonian: X-mixer applies bit-flipping between 0 and 1 on each qubit

$$\hat{H}_M = \sum_{j=1}^N X_j$$

- Loss Hamiltonian: The more adjacent nodes of different colors, the less loss

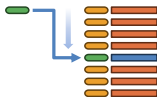
$$\hat{H}_L = \sum_{i,j \in E} Z_i Z_j$$

Conclusion

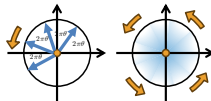
What Problems are Suitable for Quantum Computers?



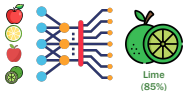
Extreme parallelization



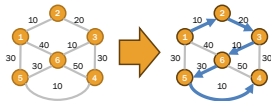
Information search



Phase estimation



Deep learning with adaptive activation functions



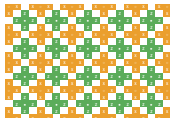
Graph-based optimization



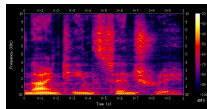
Parallelizable simulation



Secure communication and cryptography



Error correction



Spectral analysis

- Superposition allows quantum computers to perform independent tasks at the same time
- Entanglement fortifies the security of data communication
- The output will be a probability distribution of possible outputs
- **Warning:** Quantum computers work on complex matrices and eigen-decomposition

Figure 21: Quantum-suitable problems

Questions?